

Hardware MMU Architecture, Pipeline Graphs & Interface Boundaries

32-Bit Translation Architecture with Two-Tier TLB & OS Fault Hand-Off

System Design Specification

September 13, 2026

Abstract

This document provides a visual and structural specification for a 32-bit hardware Memory Management Unit (MMU). All raw Verilog source listings have been omitted in favor of comprehensive architectural block diagrams, finite state machine (FSM) graphs, and decision-tree flowcharts. A dedicated boundary specification delineates internal hardware responsibilities from external Operating System / Testbench tasks, formalizing the exact handling pipelines for segmentation faults and page faults.

Contents

1	System Hardware Architecture & Interconnect Topology	2
1.1	Address Bit Decomposition	2
1.2	System Block Diagram & Dataflow Graph	2
2	FSM Controller Execution Graph & Decision Workflows	2
2.1	Complete Finite State Machine (FSM) State Diagram	2
2.2	Address Translation Execution Flowchart (Decision Tree)	2
3	System Interface Boundaries: Hardware vs. External OS / Testbench	2
4	Detailed Handling Pipelines for System Faults	3
4.1	Segmentation Fault (<code>seg_fault</code>): Terminal Handling Flow	3
4.2	Page Fault (<code>page_fault</code>): Recoverable Handling Flow	4
4.3	Summary of Required External Drivers (Testbench Checklist)	4

1 System Hardware Architecture & Interconnect Topology

The translation pipeline converts a 32-bit Virtual Address (VA) into a 26-bit Physical Address (PA) supporting 1 KB page/frame granularity across a 64 MB physical address space.

1.1 Address Bit Decomposition

$$VA[31:0] \longrightarrow \underbrace{\text{seg_num}[6:0]}_{\text{Bits } [31:25]} \parallel \underbrace{\text{vpn}[14:0]}_{\text{Bits } [24:10]} \parallel \underbrace{\text{offset}[9:0]}_{\text{Bits } [9:0]} \quad (1)$$

1.2 System Block Diagram & Dataflow Graph

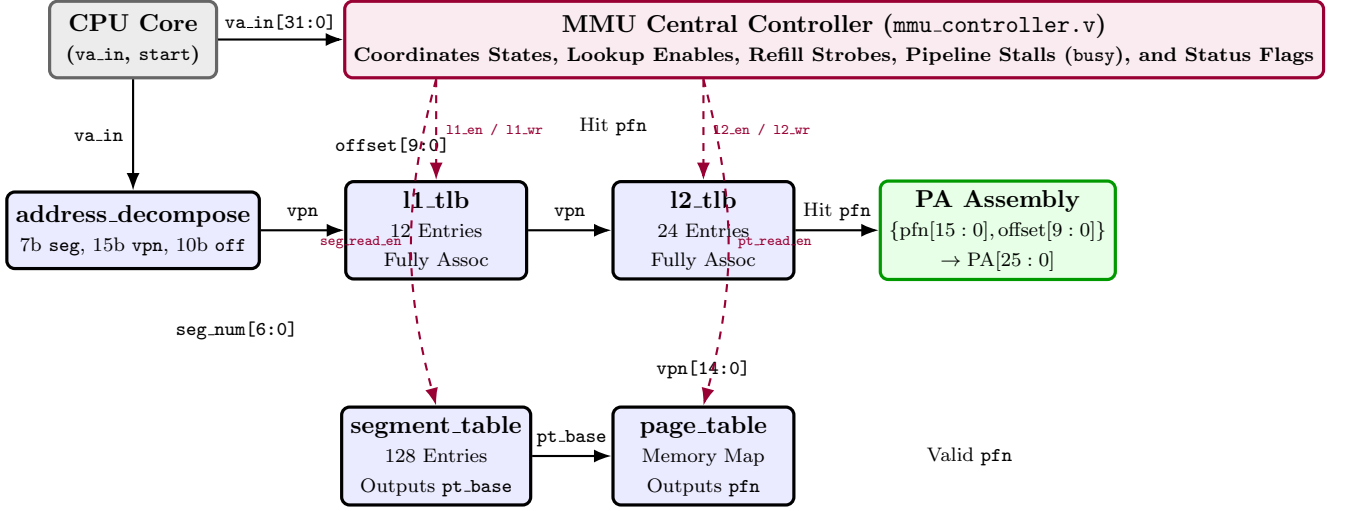


Figure 1: System Structural Block Diagram: Data Buses (solid) and FSM Controls (dashed)

2 FSM Controller Execution Graph & Decision Workflows

2.1 Complete Finite State Machine (FSM) State Diagram

The state machine uses 6 states to arbitrate cache checks and table-walk lookups.

2.2 Address Translation Execution Flowchart (Decision Tree)

3 System Interface Boundaries: Hardware vs. External OS / Testbench

The hardware MMU only **detects and signals** conditions; it cannot resolve memory faults on its own. The table below delineates the strict boundary between what is implemented in hardware versus what must be supplied by the Operating System, Testbench, or external components.

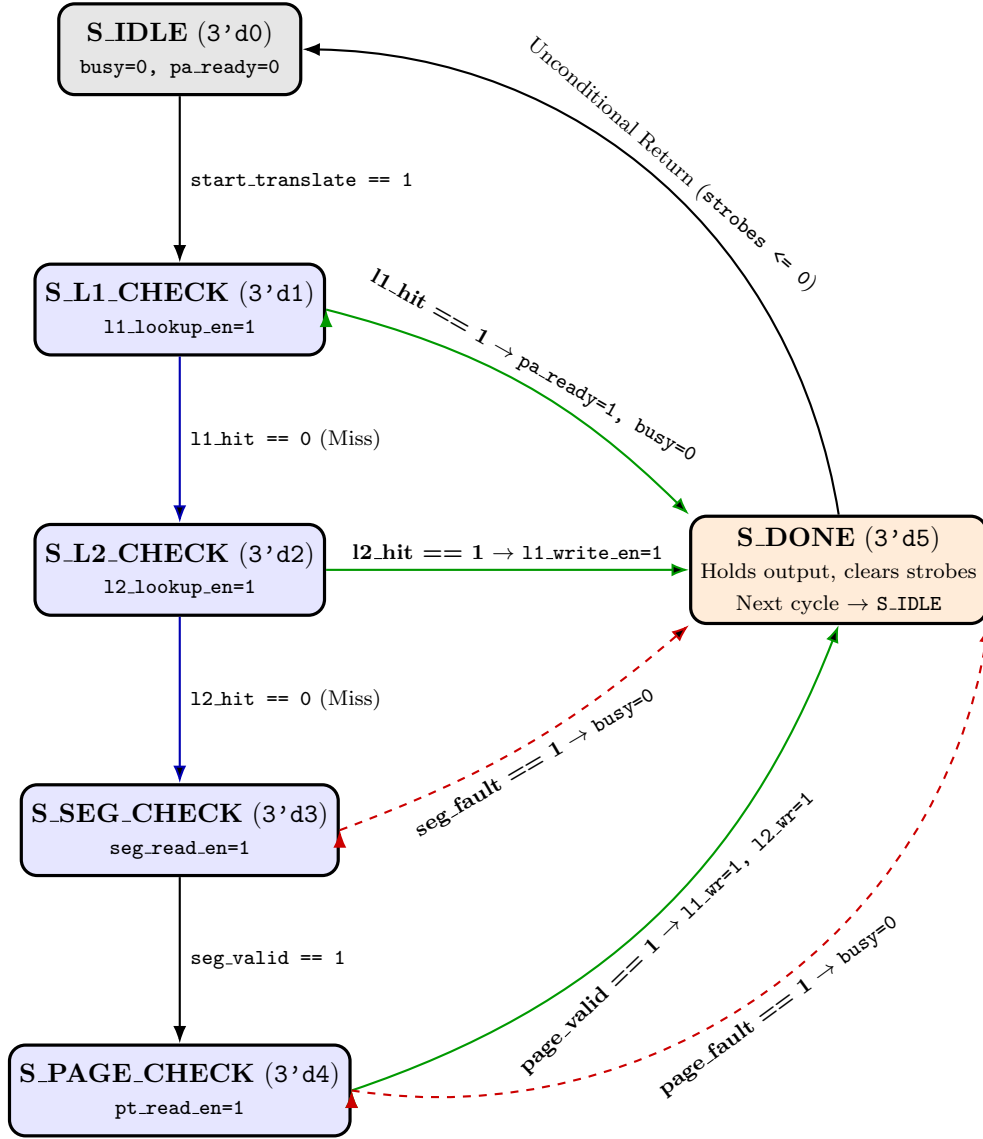


Figure 2: Controller FSM State Transitions: Blue = Progression, Green = Fast-Path Hits, Red = Fault Traps

4 Detailed Handling Pipelines for System Faults

4.1 Segmentation Fault (seg_fault): Terminal Handling Flow

A segmentation fault indicates an illegal access attempt beyond a process's allocated address space.

- **Hardware Trigger:** `valid_storage[seg_num] == 1'b0` inside `segment_table.v`.
- **Hardware Reaction:** Asserts `seg_fault <= 1'b1`, de-asserts `busy <= 1'b0`, suppresses `pa_ready`, and jumps to `S_DONE`.
- **External OS Actions Required:**
 1. Intercept the `seg_fault` interrupt signal on the system interrupt line.
 2. Read the faulting address from the latched CPU bus.
 3. Save CPU registers and call the kernel's panic/fault logging routine.
 4. Dispatch a `SIGSEGV` signal to the thread and terminate the process.
 5. Reclaim all physical frames previously assigned to this process.
 6. **Never retry or restart the instruction.**

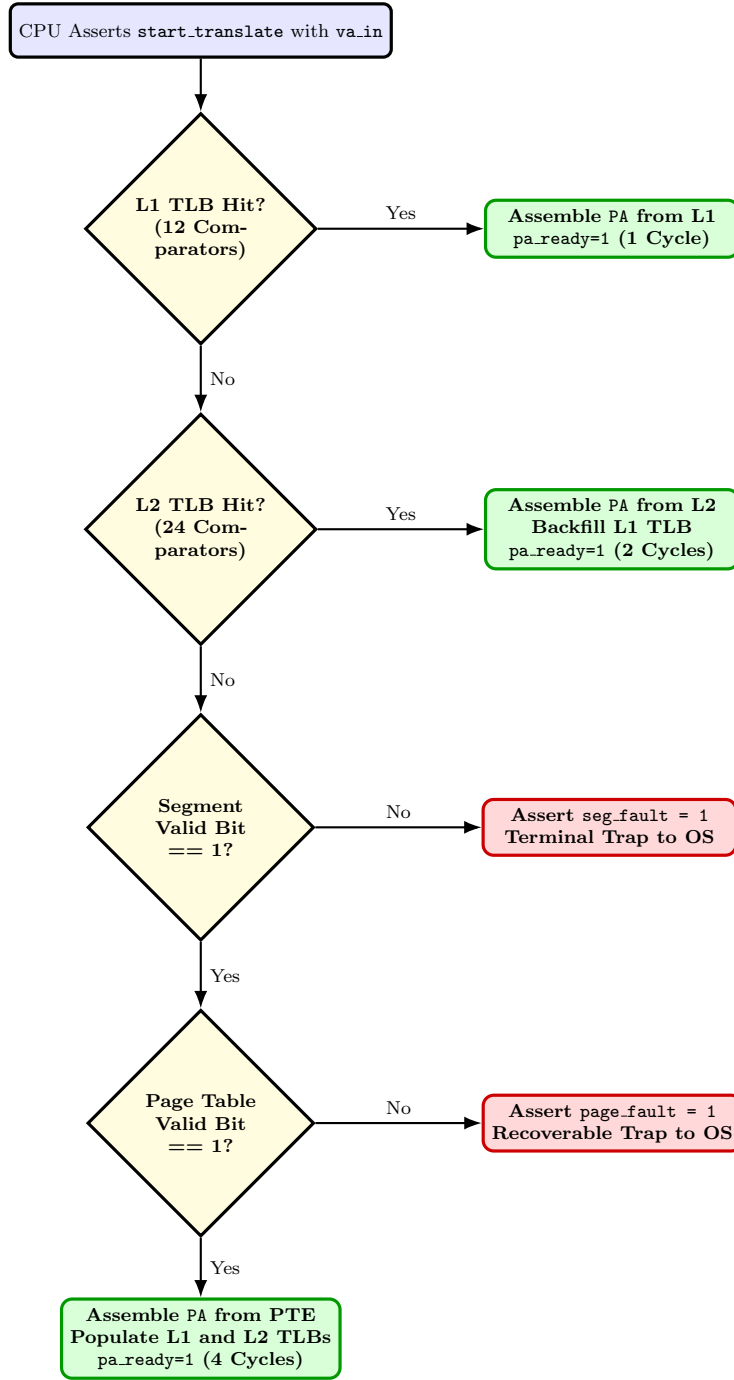


Figure 3: Step-by-Step Translation Decision Logic

4.2 Page Fault (`page_fault`): Recoverable Handling Flow

A page fault occurs when a virtual page belongs to a valid segment, but the data has not yet been loaded into physical RAM.

4.3 Summary of Required External Drivers (Testbench Checklist)

To run a complete simulation of this MMU, your testbench or companion software team must provide:

1. **Clock & Reset Generator:** Drives `clk` and active-low `rst_n`.
2. **Instruction Request Driver:** Asserts `start_translate` and presents `va_in[31:0]`, then

Functional Dimension	My Modules (Hardware Implementation)	External (OS / Testbench / CPU)
Address Partitioning	Hardware extraction of <code>seg[6:0]</code> , <code>vpn[14:0]</code> , <code>offset[9:0]</code> via combinational wiring.	Setting up virtual memory spaces and passing virtual addresses to the bus.
Cache Lookup & Refill	Parallel comparator arrays, match multiplexers, replacement pointers, synchronous refills.	Providing context switch preemption pulses (<code>preempt_pulse</code>) to invalidate tags.
Memory Table Walk	Reading segment base registers, computing <code>pt_base + vpn</code> , indexing entries.	Populating base addresses and page frame mappings in main memory tables.
Segmentation Violations	Detecting <code>valid_storage[seg] == 0</code> , asserting single-cycle <code>seg_fault = 1</code> , releasing <code>busy</code> .	Intercepting interrupt, dumping core, issuing <code>SIGSEGV</code> , killing process (Terminal).
Page Fault Exceptions	Detecting <code>valid_storage[pte] == 0</code> , asserting single-cycle <code>page_fault = 1</code> , releasing <code>busy</code> .	Allocating physical RAM frame, simulating disk read I/O, writing PTE, restarting access.
Pipeline Flow Control	Asserting <code>busy = 1</code> during translation, asserting single-cycle <code>pa_ready = 1</code> on success.	Stalling CPU dispatch while <code>busy == 1</code> ; latching <code>pa_out</code> when <code>pa_ready == 1</code> .

Table 1: System Responsibility Allocation (Hardware MMU vs. External Software/Driver)

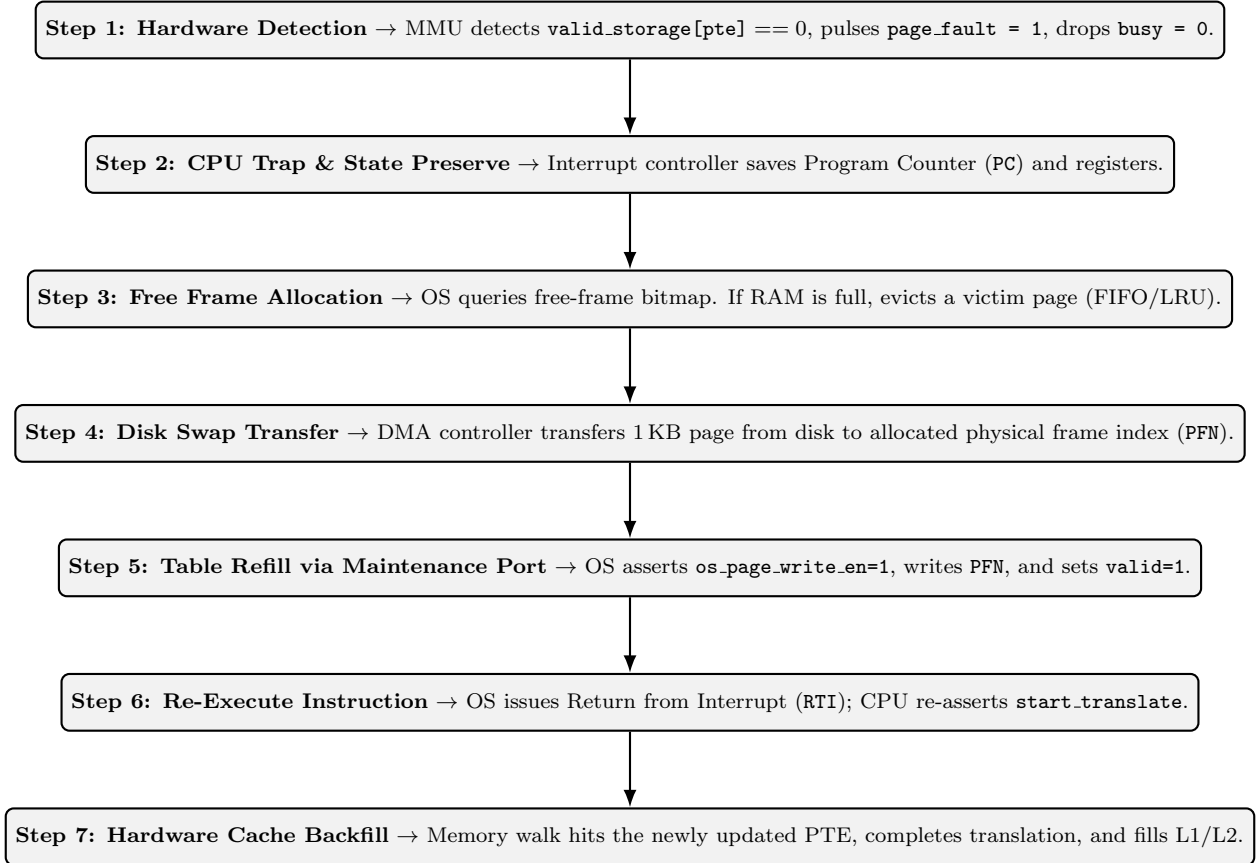


Figure 4: Complete Page Fault Recovery Cycle (Hardware Detection to Instruction Re-execution)

waits for `pa_ready`.

3. **Segmentation Fault Handler Task:** Halts simulation cleanly with an error message when

`seg_fault == 1.`

4. **Page Fault Service Task:** Emulates disk-to-memory latency, chooses an available `pfn`, updates the page table via `os_page_write_*`, and restarts the failed translation request.
5. **Context Preemption Pulse:** Asserts `preempt_pulse` whenever switching between simulated processes to flush old TLB entries.